WEEK 2 INTERNSHIP

NumPy and Pandas for Data Analytics

Task 1: Create and Explore NumPy Arrays

Aim:

To create NumPy arrays and explore their properties

Program:

```
import numpy as np

# Create a NumPy array with 10 numbers

arr = np.array([10, 20, 30, 40, 50, 60, 70, 80, 90, 100])

# Display the array

print("Original Array:")

print(arr)

# Display shape, size, and data type

print("\nShape:", arr.shape)

print("Size:", arr.size)

print("Data Type:", arr.dtype)

# Create a one-dimensional array

one_d = np.array([1, 2, 3, 4, 5])

print("\nOne-Dimensional Array:")

print(one_d)

# Create a two-dimensional array

two_d = np.array([[1, 2, 3], [4, 5, 6]])

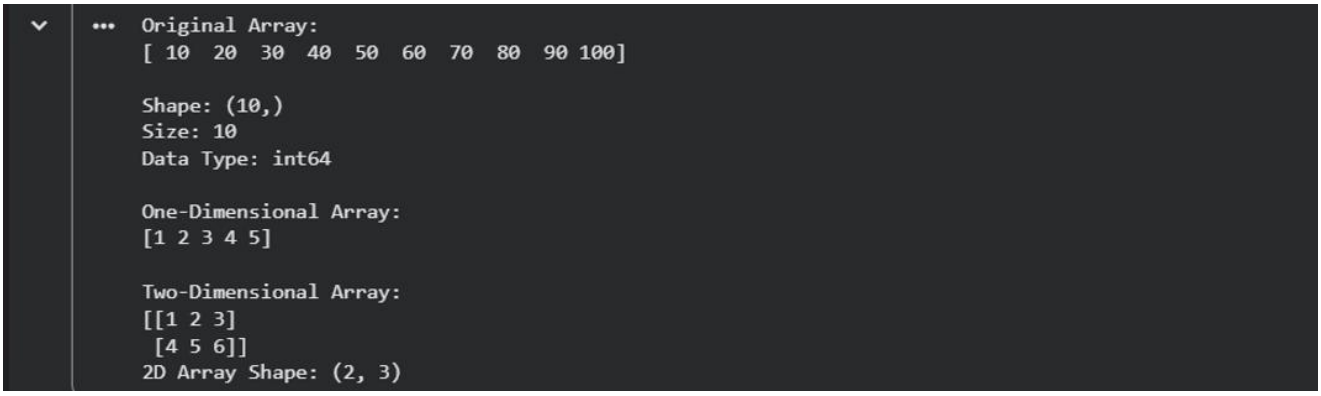
print("\nTwo-Dimensional Array:")

print(two_d)

print("2D Array Shape:", two_d.shape)
```

output:

The program creates one-dimensional and two-dimensional arrays and displays their properties.



```
Original Array:
[ 10  20  30  40  50  60  70  80  90 100]

Shape: (10,)
Size: 10
Data Type: int64

One-Dimensional Array:
[1 2 3 4 5]

Two-Dimensional Array:
[[1 2 3]
 [4 5 6]]
2D Array Shape: (2, 3)
```

Conclusion:

NumPy helps store and process numerical data efficiently.

Task 2: Array Indexing, Slicing, and Reshaping

Aim:

To access array elements and reshape an array.

Program:

```
import numpy as np

# Create a one-dimensional array

arr = np.array([10, 20, 30, 40, 50, 60])

print("Original Array:")

print(arr)

# 1. Access specific elements using indexing

print("\nElement at index 0:", arr[0])

print("Element at index 3:", arr[3])

# 2. Extract a portion using slicing

print("\nSliced Array (index 1 to 4):")

print(arr[1:5])

# 3. Create a two-dimensional array

arr2 = np.array([[1, 2, 3],

                 [4, 5, 6],

                 [7, 8, 9]])

print("\nTwo-Dimensional Array:")

print(arr2)

# 4. Access a specific row

print("\nFirst Row:")

print(arr2[0])

# 5. Access a specific column

print("\nSecond Column:")

print(arr2[:, 1])

# 6. Reshape the array
```

```
arr3 = np.arange(1, 13)

reshaped = arr3.reshape(3, 4)

print("\nOriginal Array for Reshaping:")

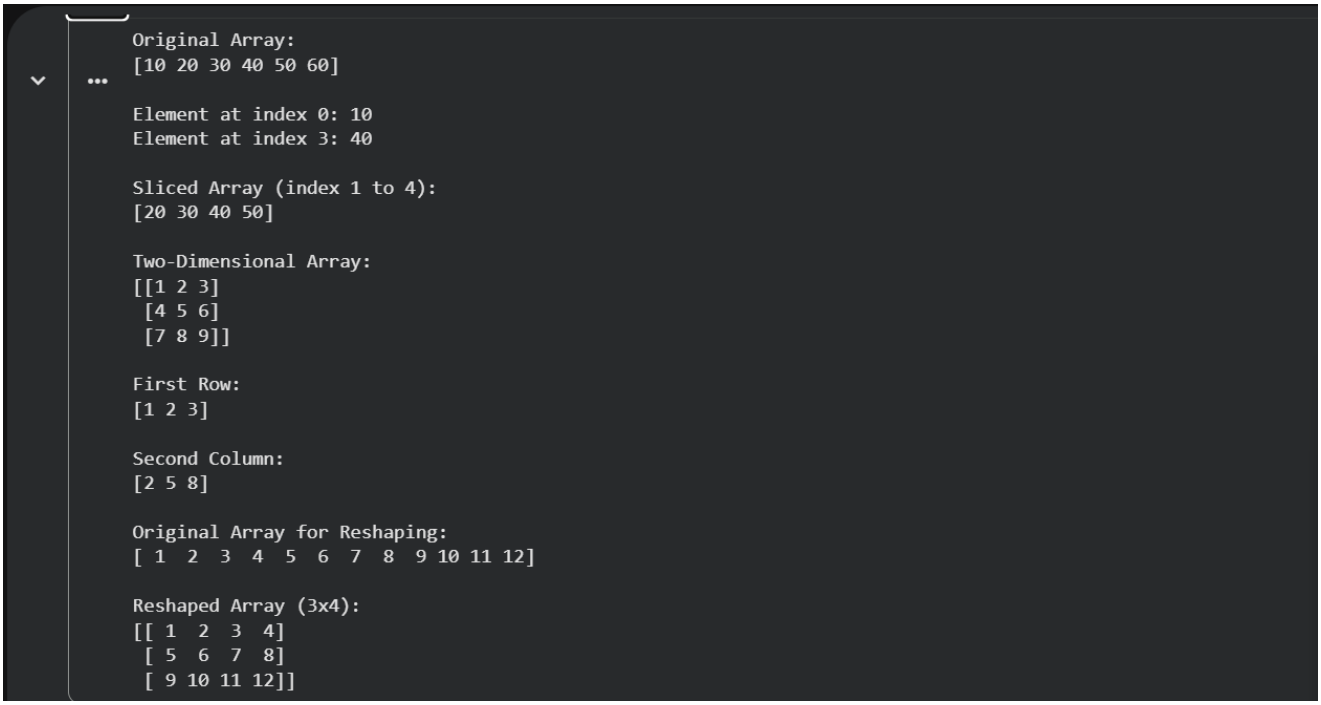
print(arr3)

print("\nReshaped Array (3x4):")

print(reshaped)
```

output:

The program accesses individual elements, selects a portion of the array, and reshapes it into two rows and three columns.



```
Original Array:
[10 20 30 40 50 60]

Element at index 0: 10
Element at index 3: 40

Sliced Array (index 1 to 4):
[20 30 40 50]

Two-Dimensional Array:
[[1 2 3]
 [4 5 6]
 [7 8 9]]

First Row:
[1 2 3]

Second Column:
[2 5 8]

Original Array for Reshaping:
[ 1  2  3  4  5  6  7  8  9 10 11 12]

Reshaped Array (3x4):
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
```

Conclusion:

Indexing, slicing, and reshaping help organize and access array data.

Task 3: Mathematical and Statistical Operations

Aim:

To perform mathematical and statistical operations using NumPy.

Program:

```
import numpy as np

# Create two numerical arrays

arr1 = np.array([10, 20, 30, 40, 50])

arr2 = np.array([2, 4, 5, 8, 10])

print("Array 1:", arr1)

print("Array 2:", arr2)

# Mathematical Operations

print("\n--- Mathematical Operations ---")

print("Addition:", np.add(arr1, arr2))

print("Subtraction:", np.subtract(arr1, arr2))

print("Multiplication:", np.multiply(arr1, arr2))

print("Division:", np.divide(arr1, arr2))

# Statistical Operations

print("\n--- Statistical Operations ---")

print("Mean:", np.mean(arr1))

print("Median:", np.median(arr1))

print("Minimum:", np.min(arr1))

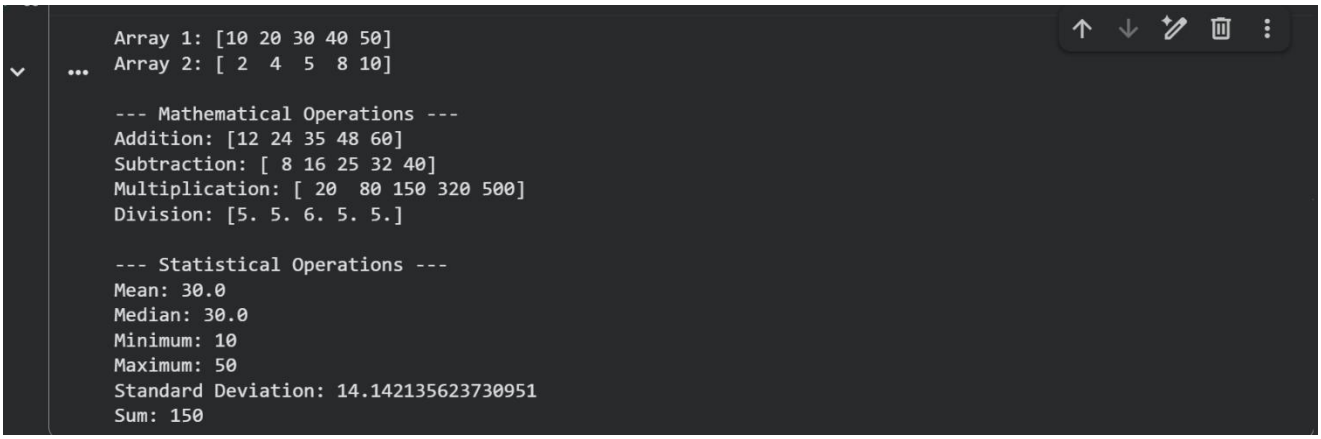
print("Maximum:", np.max(arr1))

print("Standard Deviation:", np.std(arr1))

print("Sum:", np.sum(arr1))
```

Output:

The program displays arithmetic results, sum, mean, minimum, and maximum.



Conclusion:

NumPy simplifies mathematical and statistical calculations.

Task 4: Create Pandas Series and DataFrame

Aim:

To create and display Pandas Series and DataFrame objects.

Program:

```
import pandas as pd

# Create a Pandas Series

marks = pd.Series([85, 90, 78, 92, 88])

print("Pandas Series:")

print(marks)

# Create a DataFrame containing student information

data = {

    "Name": ["Ravi", "Sita", "Arun", "Priya", "Kiran"],

    "Age": [20, 21, 19, 20, 22],

    "Marks": [85, 90, 78, 92, 88]

}

df = pd.DataFrame(data)

# Display the original DataFrame

print("\nOriginal DataFrame:")

print(df)

# Display column names

print("\nColumn Names:")

print(df.columns)

# Display index

print("\nDataFrame Index:")

print(df.index)

# Add a new column

df["Result"] = ["Pass", "Pass", "Pass", "Pass", "Pass"]
```

```
# Display the updated DataFrame

print("\nUpdated DataFrame:")

print(df)
```

output:

The program creates a Series of marks and a DataFrame containing student details.



```
Pandas Series:
0    85
1    90
2    78
3    92
4    88
dtype: int64

Original DataFrame:
    Name  Age  Marks
0   Ravi   20     85
1   Sita   21     90
2   Arun   19     78
3  Priya   20     92
4  Kiran   22     88

Column Names:
Index(['Name', 'Age', 'Marks'], dtype='object')

DataFrame Index:
RangeIndex(start=0, stop=5, step=1)

Updated DataFrame:
    Name  Age  Marks Result
0   Ravi   20     85   Pass
1   Sita   21     90   Pass
2   Arun   19     78   Pass
3  Priya   20     92   Pass
4  Kiran   22     88   Pass
```

Conclusion:

Pandas is useful for storing and managing structured data.

Task 5: Read and Inspect a CSV File

Aim:

To read and inspect data from a CSV file.

Program:

```
import pandas as pd

# Create a sample dataset
```

```
data = {
    "Name": ["Ravi", "Sita", "Arun", "Priya", "Kiran",
             "Meena", "Rahul", "Divya"],
    "Age": [20, 21, 19, 20, 22, 21, 20, 19],
    "Marks": [85, 90, 78, 92, 88, 76, 95, 84],
    "Department": ["CSE", "ECE", "CSE", "IT", "ECE",
                   "IT", "CSE", "IT"]
}
df = pd.DataFrame(data)
# Save the dataset as a CSV file
df.to_csv("student_data.csv", index=False)
# Read the CSV file using Pandas
df = pd.read_csv("student_data.csv")
# Display the first 5 rows
print("First 5 Rows:")
print(df.head())
# Display the last 5 rows
print("\nLast 5 Rows:")
print(df.tail())
# Check the number of rows and columns
print("\nDataset Shape:")
print(df.shape)
# Display column names
print("\nColumn Names:")
print(df.columns.tolist())
# Check data types
print("\nData Types:")
print(df.dtypes)
```

```
# Display information about the dataset

print("\nDataset Information:")

df.info()

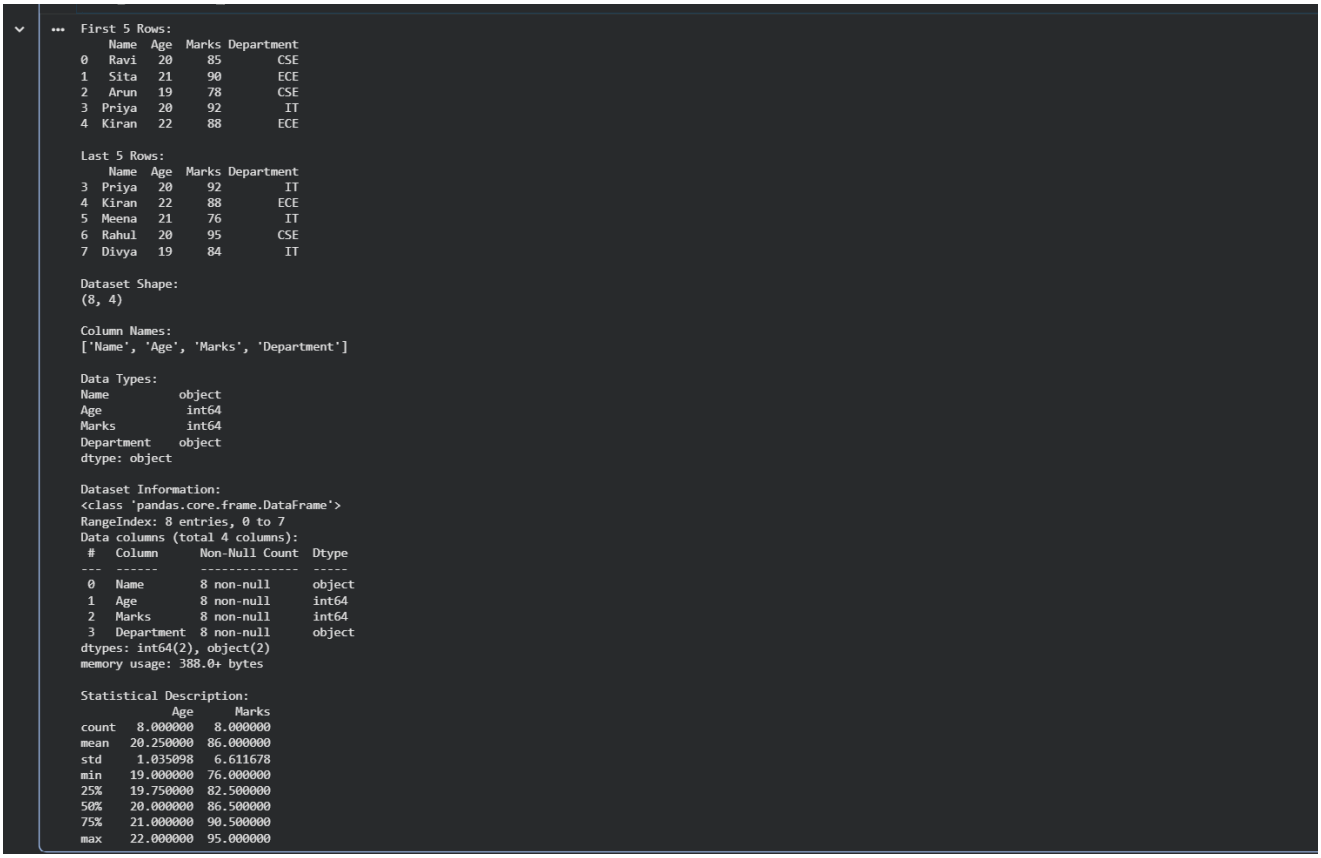
# Display statistical description

print("\nStatistical Description:")

print(df.describe())
```

ouput:

The program reads students.csv and displays sample rows, column names, dataset information, and a summary.



```
First 5 Rows:
    Name  Age  Marks Department
0   Ravi   20     85        CSE
1   Sita   21     90        ECE
2   Arun   19     78        CSE
3  Priya   20     92         IT
4  Kiran   22     88        ECE

Last 5 Rows:
    Name  Age  Marks Department
3  Priya   20     92         IT
4  Kiran   22     88        ECE
5  Meena   21     76         IT
6  Rahul   20     95        CSE
7  Divya   19     84         IT

Dataset Shape:
(8, 4)

Column Names:
['Name', 'Age', 'Marks', 'Department']

Data Types:
Name          object
Age            int64
Marks          int64
Department    object
dtype: object

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8 entries, 0 to 7
Data columns (total 4 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   Name        8 non-null      object
 1   Age         8 non-null      int64
 2   Marks       8 non-null      int64
 3   Department  8 non-null      object
dtypes: int64(2), object(2)
memory usage: 388.0+ bytes

Statistical Description:
             Age      Marks
count   8.000000   8.000000
mean   20.250000  86.000000
std     1.035098   6.611678
min    19.000000  76.000000
25%    19.750000  82.500000
50%    20.000000  86.500000
75%    21.000000  90.500000
max    22.000000  95.000000
```

Conclusion

Pandas makes reading and inspecting CSV data easy.

Task 6: Select, Filter, and Sort Data

Aim:

To select columns, filter records, and sort data.

Program:

```
import pandas as pd

# Create a sample dataset

data = {

    "Name": ["Ravi", "Sita", "Arun", "Priya", "Kiran",

             "Meena", "Rahul", "Divya"],

    "Age": [20, 21, 19, 20, 22, 21, 20, 19],

    "Marks": [85, 90, 78, 92, 88, 76, 95, 84],

    "Department": ["CSE", "ECE", "CSE", "IT", "ECE",

                   "IT", "CSE", "IT"]

}

df = pd.DataFrame(data)

# 1. Select a single column

print("Student Names:")

print(df["Name"])

# 2. Select multiple columns

print("\nSelected Columns (Name and Marks):")

print(df[["Name", "Marks"]])

# 3. Filter students with marks greater than 85

print("\nStudents Scoring Above 85:")

print(df[df["Marks"] > 85])

# 4. Filter students belonging to CSE

print("\nCSE Department Students:")

print(df[df["Department"] == "CSE"])
```

```
# 5. Filter students aged 20 or above

print("\nStudents Aged 20 or Above:")

print(df[df["Age"] >= 20])

# 6. Sort students by marks in ascending order

print("\nMarks Sorted in Ascending Order:")

print(df.sort_values(by="Marks"))

# 7. Sort students by marks in descending order

print("\nMarks Sorted in Descending Order:")

print(df.sort_values(by="Marks", ascending=False))
```

Ouput:

The program selects the Name column, filters students scoring at least 80, and sorts marks from highest to lowest.



```
Student Names:
0    Ravi
1    Sita
2    Arun
3   Priya
4   Kiran
5   Meena
6   Rahul
7   Divya
Name: Name, dtype: object

Selected Columns (Name and Marks):
    Name  Marks
0   Ravi     85
1   Sita     90
2   Arun     78
3  Priya     92
4  Kiran     88
5  Meena     76
6  Rahul     95
7  Divya     84

Students Scoring Above 85:
    Name  Age  Marks Department
1   Sita   21     90        ECE
3  Priya   20     92         IT
4  Kiran   22     88        ECE
6  Rahul   20     95        CSE

CSE Department Students:
    Name  Age  Marks Department
0   Ravi   20     85        CSE
2   Arun   19     78        CSE
6  Rahul   20     95        CSE

Students Aged 20 or Above:
    Name  Age  Marks Department
0   Ravi   20     85        CSE
1   Sita   21     90        ECE
3  Priya   20     92         IT
4  Kiran   22     88        ECE
5  Meena   21     76         IT
6  Rahul   20     95        CSE

Marks Sorted in Ascending Order:
    Name  Age  Marks Department
5  Meena   21     76         IT
2   Arun   19     78        CSE
7  Divya   19     84         IT
0   Ravi   20     85        CSE
4  Kiran   22     88        ECE
1   Sita   21     90        ECE
3  Priya   20     92         IT
6  Rahul   20     95        CSE

Marks Sorted in Descending Order:
    Name  Age  Marks Department
6  Rahul   20     95        CSE
```

Conclusion:

Filtering and sorting help identify and organize useful information.

Task 7: Handle Missing Values

Aim:

To identify and handle missing values in a DataFrame.

Program:

```
import pandas as pd

import numpy as np

# Create a dataset with missing values

data = {

    "Name": ["Ravi", "Sita", "Arun", "Priya", "Kiran"],

    "Age": [20, 21, np.nan, 20, 22],

    "Marks": [85, np.nan, 78, 92, 88],

    "Department": ["CSE", "ECE", "CSE", None, "ECE"]

}

df = pd.DataFrame(data)

print("Original Dataset:")

print(df)

# 1. Identify missing values

print("\nMissing Values:")

print(df.isnull())

# 2. Count missing values in each column

print("\nMissing Values Count:")

print(df.isnull().sum())


# 3. Remove rows containing missing values

df_dropped = df.dropna()

print("\nDataset After Removing Missing Values:")

print(df_dropped)

# 4. Fill missing numerical values with column means
```

```
df_filled = df.copy()

df_filled["Age"] = df_filled["Age"].fillna(df_filled["Age"].mean())

df_filled["Marks"] = df_filled["Marks"].fillna(

    df_filled["Marks"].mean()

)

# 5. Fill missing categorical values

df_filled["Department"] = df_filled["Department"].fillna("Unknown")

print("\nDataset After Filling Missing Values:")

print(df_filled)

# 6. Verify missing values after filling

print("\nMissing Values After Filling:")

print(df_filled.isnull().sum())
```

output:

The program identifies missing values and fills missing marks with the mean and missing city names with Unknown.

```
Original Dataset:
     Name   Age  Marks Department
0   Ravi  20.0   85.0        CSE
1   Sita  21.0    NaN        ECE
2   Arun   NaN   78.0        CSE
3  Priya  20.0   92.0       None
4  Kiran  22.0   88.0        ECE

Missing Values:
    Name    Age  Marks  Department
0  False  False  False       False
1  False  False   True       False
2  False   True  False       False
3  False  False  False        True
4  False  False  False       False

Missing Values Count:
Name          0
Age           1
Marks         1
Department    1
dtype: int64

Dataset After Removing Missing Values:
    Name   Age  Marks Department
0   Ravi  20.0   85.0        CSE
4  Kiran  22.0   88.0        ECE

Dataset After Filling Missing Values:
    Name    Age  Marks Department
0   Ravi  20.00  85.00        CSE
1   Sita  21.00  85.75        ECE
2   Arun  20.75  78.00        CSE
3  Priya  20.00  92.00    Unknown
4  Kiran  22.00  88.00        ECE

Missing Values After Filling:
Name          0
Age           0
Marks         0
Department    0
dtype: int64
```

Conclusion:

Handling missing values improves data quality.

Task 8: Merge, Concatenate, Group, and Pivot

Aim:

To combine and summarize datasets using Pandas.

Program:

```
import pandas as pd

# Create student information

students = pd.DataFrame({

    "Student_ID": [101, 102, 103, 104, 105],

    "Name": ["Ravi", "Sita", "Arun", "Priya", "Kiran"],
```

```
    "Department": ["CSE", "ECE", "CSE", "ECE", "CSE"]
})
# Create student marks
marks = pd.DataFrame({
    "Student_ID": [101, 102, 103, 104, 105],
    "Marks": [85, 90, 78, 92, 88],
    "Semester": [4, 4, 4, 4, 4]
})
# 1. Merge two DataFrames
merged_df = pd.merge(students, marks, on="Student_ID")
print("1. Merged DataFrame:")
print(merged_df)
# 2. Concatenate two DataFrames
df1 = pd.DataFrame({
    "Name": ["Ravi", "Sita"],
    "Marks": [85, 90]
})
df2 = pd.DataFrame({
    "Name": ["Arun", "Priya"],
    "Marks": [78, 92]
})

concatenated_df = pd.concat([df1, df2], ignore_index=True)
print("\n2. Concatenated DataFrame:")
print(concatenated_df)
# 3. GroupBy Department and calculate average marks
grouped_df = merged_df.groupby("Department")["Marks"].mean()
print("\n3. Average Marks by Department:")
```

```
print(grouped_df)

# 4. Create a Pivot Table

pivot_df = pd.pivot_table(

    merged_df,

    values="Marks",

    index="Department",

    columns="Semester",

    aggfunc="mean"

)

print("\n4. Pivot Table:")

print(pivot_df)
```

ouput:

The program merges and concatenates tables and calculates department-wise average marks.

```
1. Merged DataFrame:
   Student_ID   Name Department  Marks  Semester
0         101   Ravi        CSE     85         4
1         102   Sita        ECE     90         4
2         103   Arun        CSE     78         4
3         104  Priya        ECE     92         4
4         105  Kiran        CSE     88         4

2. Concatenated DataFrame:
    Name  Marks
0   Ravi     85
1   Sita     90
2   Arun     78
3  Priya     92

3. Average Marks by Department:
Department
CSE    83.666667
ECE    91.000000
Name: Marks, dtype: float64

4. Pivot Table:
Semester           4
Department
CSE         83.666667
ECE         91.000000
```

Conclusion:

Pandas provides useful functions for combining and summarizing data.

Task 9: Export Data to CSV and Excel

Aim:

To save DataFrame data in CSV and Excel formats.

Program:

```
import pandas as pd

# Create a sample DataFrame

data = {

    "Name": ["Ravi", "Sita", "Arun", "Priya", "Kiran"],

    "Age": [20, 21, 19, 20, 22],

    "Marks": [85, 90, 78, 92, 88],

    "Department": ["CSE", "ECE", "CSE", "IT", "ECE"]

}

df = pd.DataFrame(data)

print("Original DataFrame:")

print(df)

# Export DataFrame to CSV

df.to_csv("student_data.csv", index=False)

print("\nData exported successfully to student_data.csv")

# Export DataFrame to Excel

df.to_excel("student_data.xlsx", index=False)

print("Data exported successfully to student_data.xlsx")

# Read the exported CSV file

csv_data = pd.read_csv("student_data.csv")

print("\nData read from CSV file:")

print(csv_data)

# Read the exported Excel file

excel_data = pd.read_excel("student_data.xlsx")

print("\nData read from Excel file:")
```

```
print(excel_data)

# Display the shape of exported data

print("\nCSV Data Shape:", csv_data.shape)

print("Excel Data Shape:", excel_data.shape)
```

ouput:

The program creates student_results.csv and student_results.xlsx in the current working folder.

```
Original DataFrame:
    Name  Age  Marks Department
0   Ravi   20     85        CSE
1   Sita   21     90        ECE
2   Arun   19     78        CSE
3  Priya   20     92         IT
4  Kiran   22     88        ECE

Data exported successfully to student_data.csv
Data exported successfully to student_data.xlsx

Data read from CSV file:
    Name  Age  Marks Department
0   Ravi   20     85        CSE
1   Sita   21     90        ECE
2   Arun   19     78        CSE
3  Priya   20     92         IT
4  Kiran   22     88        ECE

Data read from Excel file:
    Name  Age  Marks Department
0   Ravi   20     85        CSE
1   Sita   21     90        ECE
2   Arun   19     78        CSE
3  Priya   20     92         IT
4  Kiran   22     88        ECE

CSV Data Shape: (5, 4)
Excel Data Shape: (5, 4)
```

Conclusion:

Exporting allows data to be saved, shared, and opened in spreadsheet applications.

Task 10: Mini Project — Student Performance Analysis

Aim:

To analyse student marks and calculate totals, averages, and results.

Program:

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

# Step 1: Create a student performance dataset

data = {

    "Name": ["Ravi", "Sita", "Arun", "Priya", "Kiran",

             "Meena", "Rahul", "Divya", "Vijay", "Anu"],

    "Department": ["CSE", "ECE", "CSE", "IT", "ECE",

                   "IT", "CSE", "IT", "ECE", "CSE"],

    "Marks": [85, 90, 78, 92, 88, 76, 95, 84, 81, 89],

    "Attendance": [90, 95, 80, 98, 88, 75, 96, 85, 82, 91]

}

df = pd.DataFrame(data)

# Step 2: Display the dataset

print("STUDENT PERFORMANCE DATASET")

print(df.to_string(index=False))

# Step 3: Calculate statistical information

print("\nSTATISTICAL ANALYSIS")

print("Average Marks:", df["Marks"].mean())

print("Highest Marks:", df["Marks"].max())

print("Lowest Marks:", df["Marks"].min())

print("Median Marks:", df["Marks"].median())

print("Average Attendance:", df["Attendance"].mean()) # Step 4: Identify the top-performing
student
```

```
top_student = df.loc[df["Marks"].idxmax()]

print("\nTOP-PERFORMING STUDENT")

print("Name:", top_student["Name"])

print("Department:", top_student["Department"])

print("Marks:", top_student["Marks"])

# Step 5: Calculate average marks by department

print("\nDEPARTMENT-WISE AVERAGE MARKS")

department_avg = df.groupby("Department")["Marks"].mean()

print(department_avg.round(2))

# Step 6: Filter students scoring 85 or above

print("\nSTUDENTS SCORING 85 OR ABOVE")

high_scorers = df[df["Marks"] >= 85]

print(high_scorers.to_string(index=False))

# Step 7: Sort students by marks

print("\nSTUDENTS SORTED BY MARKS")

sorted_df = df.sort_values(by="Marks", ascending=False)

print(sorted_df.to_string(index=False))

# Step 8: Create a bar chart

department_avg.plot(kind="bar", title="Average Marks by Department")

plt.xlabel("Department")

plt.ylabel("Average Marks")

plt.tight_layout()

plt.show()

# Step 9: Write the analysis conclusion

print("\nCONCLUSION")

print("The average student mark is:", round(df["Marks"].mean(), 2))

print("The highest-scoring student is:", top_student["Name"])

print("The department with the highest average marks is:",
```

```
department_avg.idxmax())
```

Output:

The program displays student totals, averages, result labels, department-wise averages, and the overall class average.

```
STUDENT PERFORMANCE DATASET
    Name Department  Marks  Attendance
    Ravi        CSE     85          90
    Sita        ECE     90          95
    Arun        CSE     78          80
   Priya         IT     92          98
   Kiran        ECE     88          88
   Meena         IT     76          75
   Rahul        CSE     95          96
   Divya         IT     84          85
   Vijay        ECE     81          82
     Anu        CSE     89          91

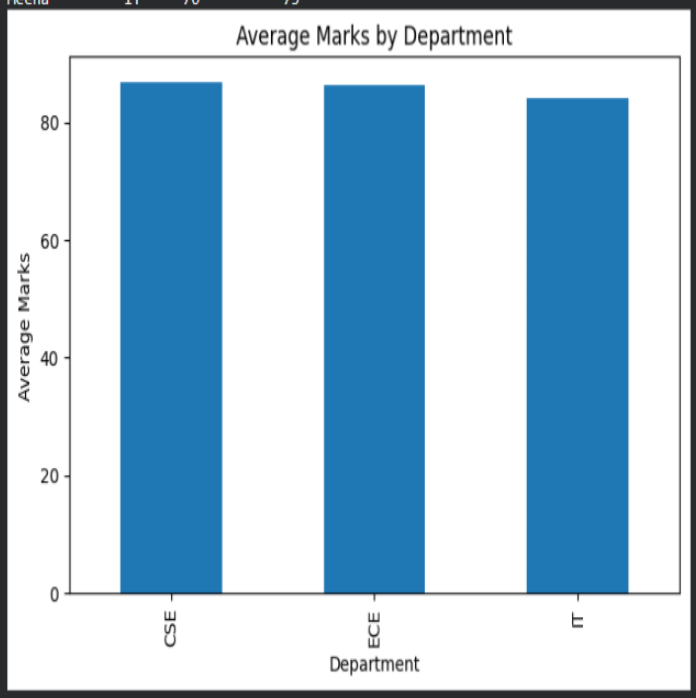
STATISTICAL ANALYSIS
Average Marks: 85.8
Highest Marks: 95
Lowest Marks: 76
Median Marks: 86.5
Average Attendance: 88.0

TOP-PERFORMING STUDENT
Name: Rahul
Department: CSE
Marks: 95

DEPARTMENT-WISE AVERAGE MARKS
Department
CSE    86.75
ECE    86.33
IT     84.00
Name: Marks, dtype: float64

STUDENTS SCORING 85 OR ABOVE
    Name Department  Marks  Attendance
    Ravi        CSE     85          90
    Sita        ECE     90          95
   Priya         IT     92          98
   Kiran        ECE     88          88
   Rahul        CSE     95          96
     Anu        CSE     89          91
```

```
STUDENTS SORTED BY MARKS
  Name Department  Marks  Attendance
Rahul        CSE     95          96
Priya         IT     92          98
 Sita        ECE     90          95
  Anu        CSE     89          91
Kiran        ECE     88          88
 Ravi        CSE     85          90
Divya         IT     84          85
Vijay        ECE     81          82
 Arun        CSE     78          80
Meena         IT     76          75
```




```
CONCLUSION
The average student mark is: 85.8
The highest-scoring student is: Rahul
The department with the highest average marks is: CSE
```

Conclusion:

Pandas helps analyse student performance and summarize academic data.